

# Cancer Classification Project: Supervised Learning

## Introduction

This mini project focuses on applying a variety of supervised learning algorithms to a real-world classification task. The goal is to predict whether a breast tumor is malignant or benign using a dataset of diagnostic features extracted from digitized images of fine needle aspirate (FNA) biopsies. The dataset was selected from a well-organized Kaggle repository, making it ideal for structured tabular analysis. It contains 30 numeric features for 569 observations, satisfying the recommendation to avoid overly simple or over complex data.

The analysis includes:

- Exploratory Data Analysis (EDA) to visualize class distribution, detect potential outliers, and assess correlations between features
- Preprocessing steps such as feature scaling, label encoding, and handling necessary columns
- Implementation and evaluation of multiple classification algorithms with performance metrics including accuracy, F1 score, and ROC AUC
- Feature importance analysis for interpretability, and a discussion of model performance tradeoffs

## Section 1: Load and Inspect Data

```
In [22]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# load dataset
data = pd.read_csv('Cancer_Data.csv')

# display first few rows
data.head()

# check shape and data types
data.info()

# drop unnecessary columns
data.drop(columns=['id', 'Unnamed: 32'], inplace=True, errors='ignore')

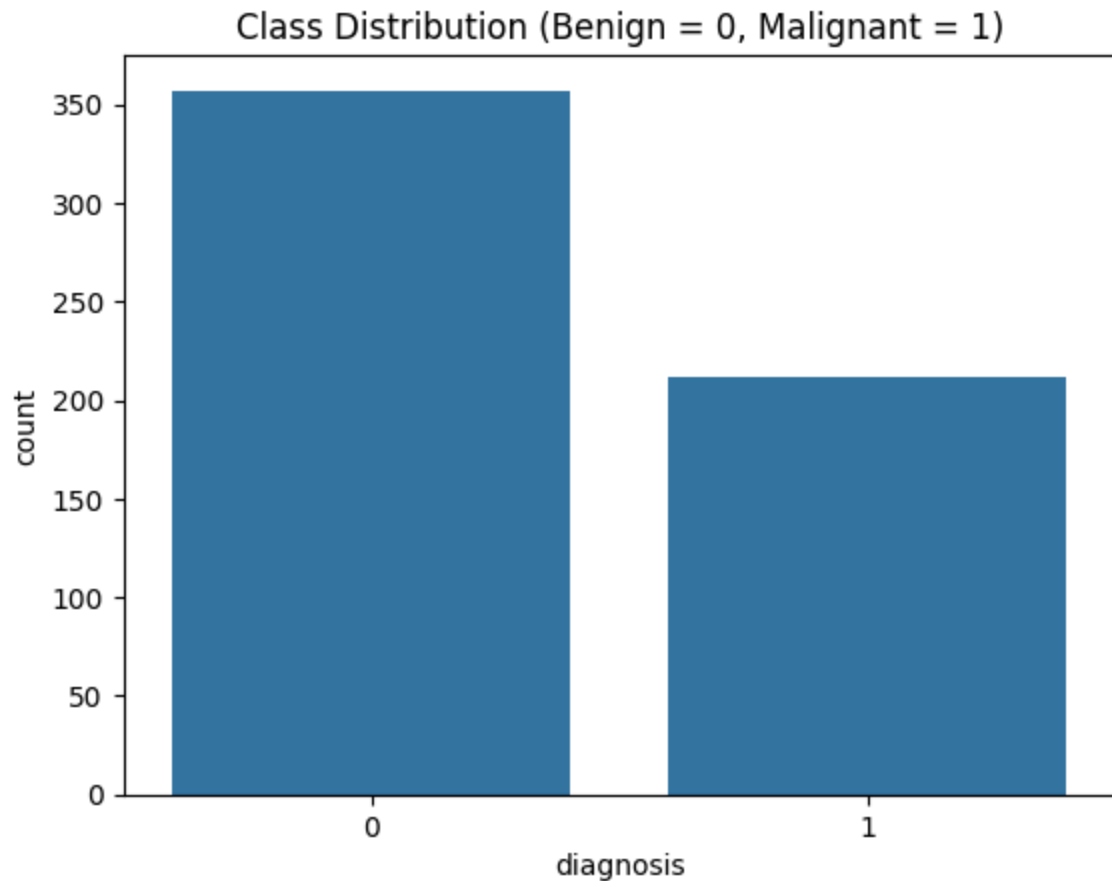
# check for null values
data.isnull().sum()

# encode the diagnosis label (B=0, M=1)
data['diagnosis'] = data['diagnosis'].map({'B': 0, 'M': 1})
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                     569 non-null    int64
1   diagnosis                             569 non-null    object
2   radius_mean                           569 non-null    float64
3   texture_mean                           569 non-null    float64
4   perimeter_mean                         569 non-null    float64
5   area_mean                             569 non-null    float64
6   smoothness_mean                       569 non-null    float64
7   compactness_mean                      569 non-null    float64
8   concavity_mean                        569 non-null    float64
9   concave points_mean                   569 non-null    float64
10  symmetry_mean                         569 non-null    float64
11  fractal_dimension_mean                 569 non-null    float64
12  radius_se                              569 non-null    float64
13  texture_se                             569 non-null    float64
14  perimeter_se                           569 non-null    float64
15  area_se                                569 non-null    float64
16  smoothness_se                          569 non-null    float64
17  compactness_se                         569 non-null    float64
18  concavity_se                           569 non-null    float64
19  concave points_se                      569 non-null    float64
20  symmetry_se                            569 non-null    float64
21  fractal_dimension_se                   569 non-null    float64
22  radius_worst                           569 non-null    float64
23  texture_worst                           569 non-null    float64
24  perimeter_worst                        569 non-null    float64
25  area_worst                             569 non-null    float64
26  smoothness_worst                       569 non-null    float64
27  compactness_worst                      569 non-null    float64
28  concavity_worst                        569 non-null    float64
29  concave points_worst                   569 non-null    float64
30  symmetry_worst                         569 non-null    float64
31  fractal_dimension_worst                 569 non-null    float64
32  Unnamed: 32                            0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

## Section 2: Exploratory Data Analysis (EDA)

```
In [7]: # class distribution
sns.countplot(x='diagnosis', data=data)
plt.title('Class Distribution (Benign = 0, Malignant = 1)')
plt.show()
```



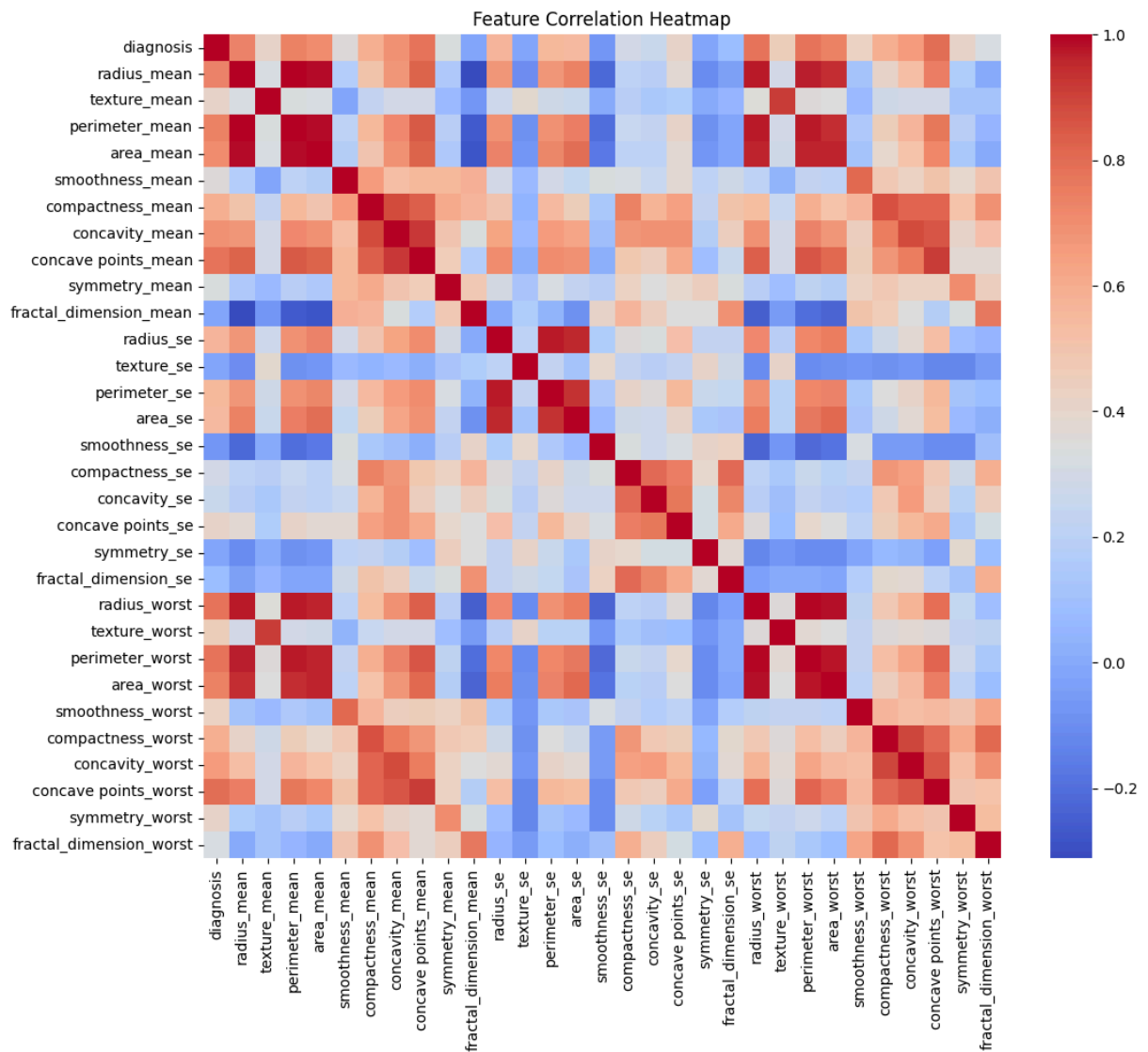
In [8]: `# summary statistics`  
`data.describe()`

Out[8]:

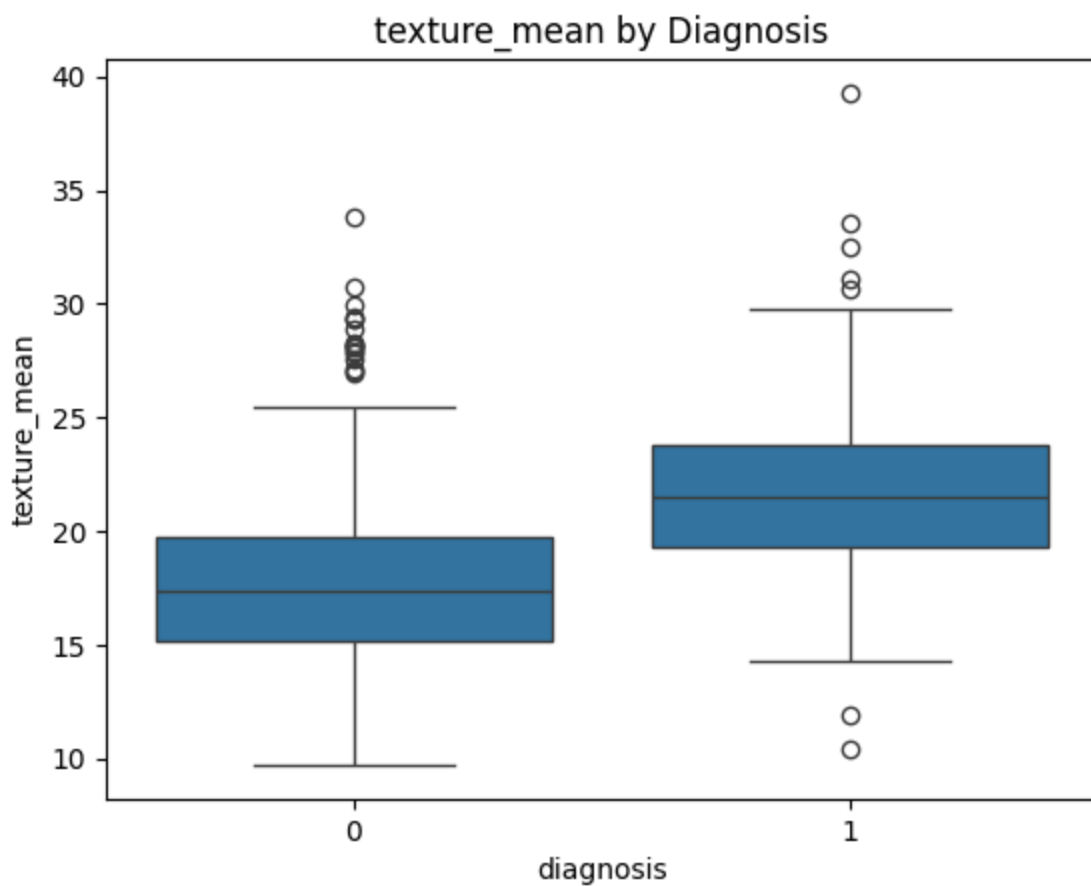
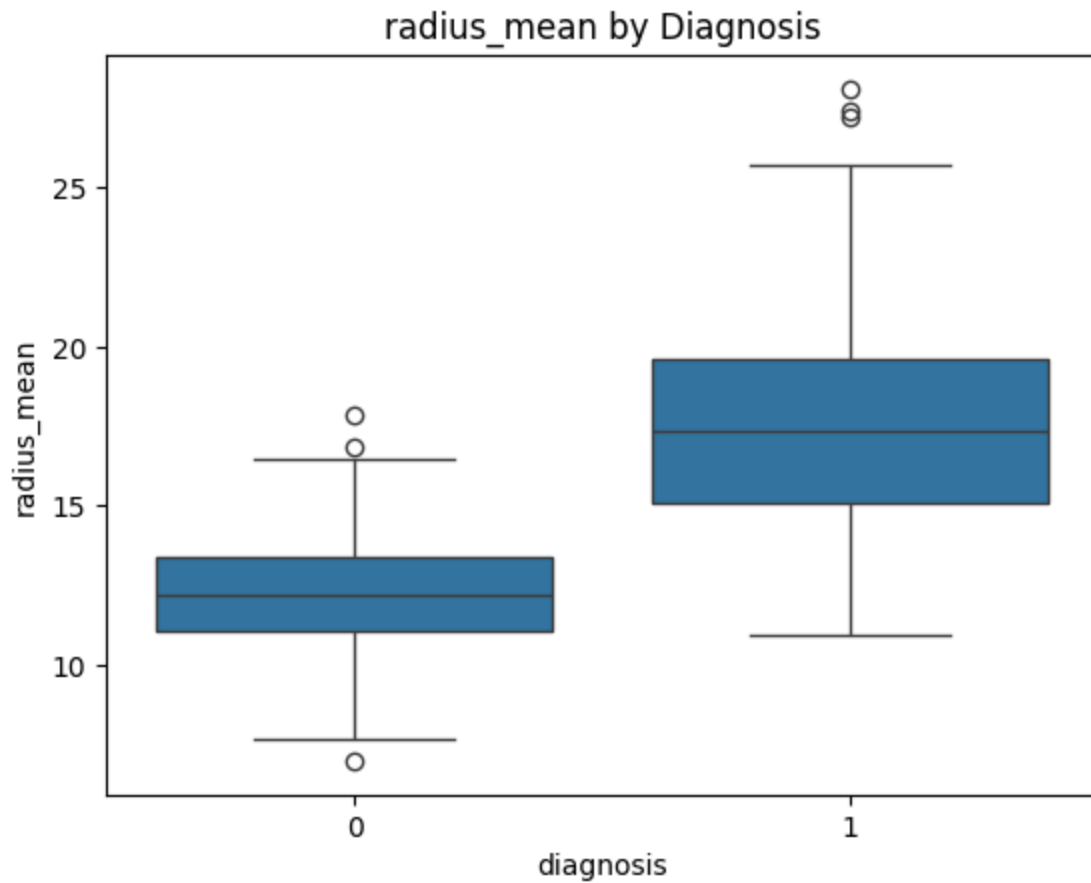
|              | diagnosis  | radius_mean | texture_mean | perimeter_mean | area_mean   | smoot |
|--------------|------------|-------------|--------------|----------------|-------------|-------|
| <b>count</b> | 569.000000 | 569.000000  | 569.000000   | 569.000000     | 569.000000  |       |
| <b>mean</b>  | 0.372583   | 14.127292   | 19.289649    | 91.969033      | 654.889104  |       |
| <b>std</b>   | 0.483918   | 3.524049    | 4.301036     | 24.298981      | 351.914129  |       |
| <b>min</b>   | 0.000000   | 6.981000    | 9.710000     | 43.790000      | 143.500000  |       |
| <b>25%</b>   | 0.000000   | 11.700000   | 16.170000    | 75.170000      | 420.300000  |       |
| <b>50%</b>   | 0.000000   | 13.370000   | 18.840000    | 86.240000      | 551.100000  |       |
| <b>75%</b>   | 1.000000   | 15.780000   | 21.800000    | 104.100000     | 782.700000  |       |
| <b>max</b>   | 1.000000   | 28.110000   | 39.280000    | 188.500000     | 2501.000000 |       |

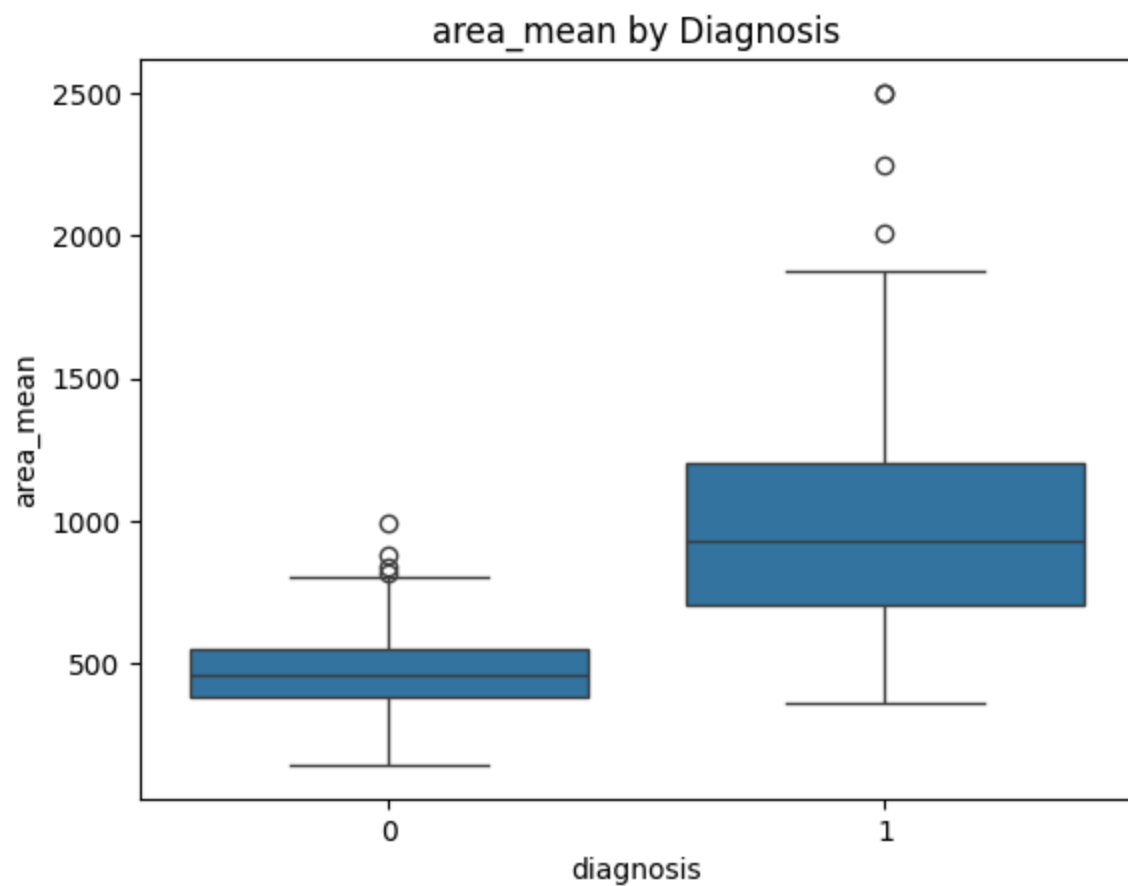
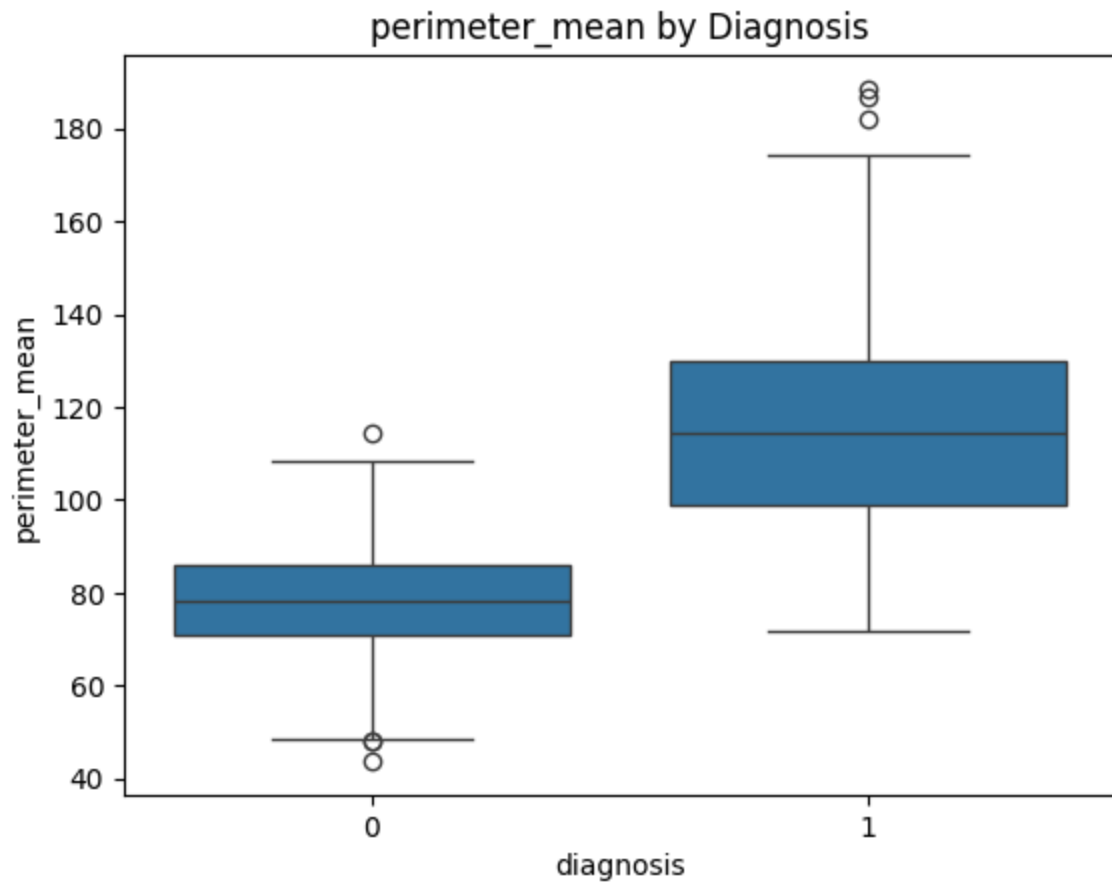
8 rows x 31 columns

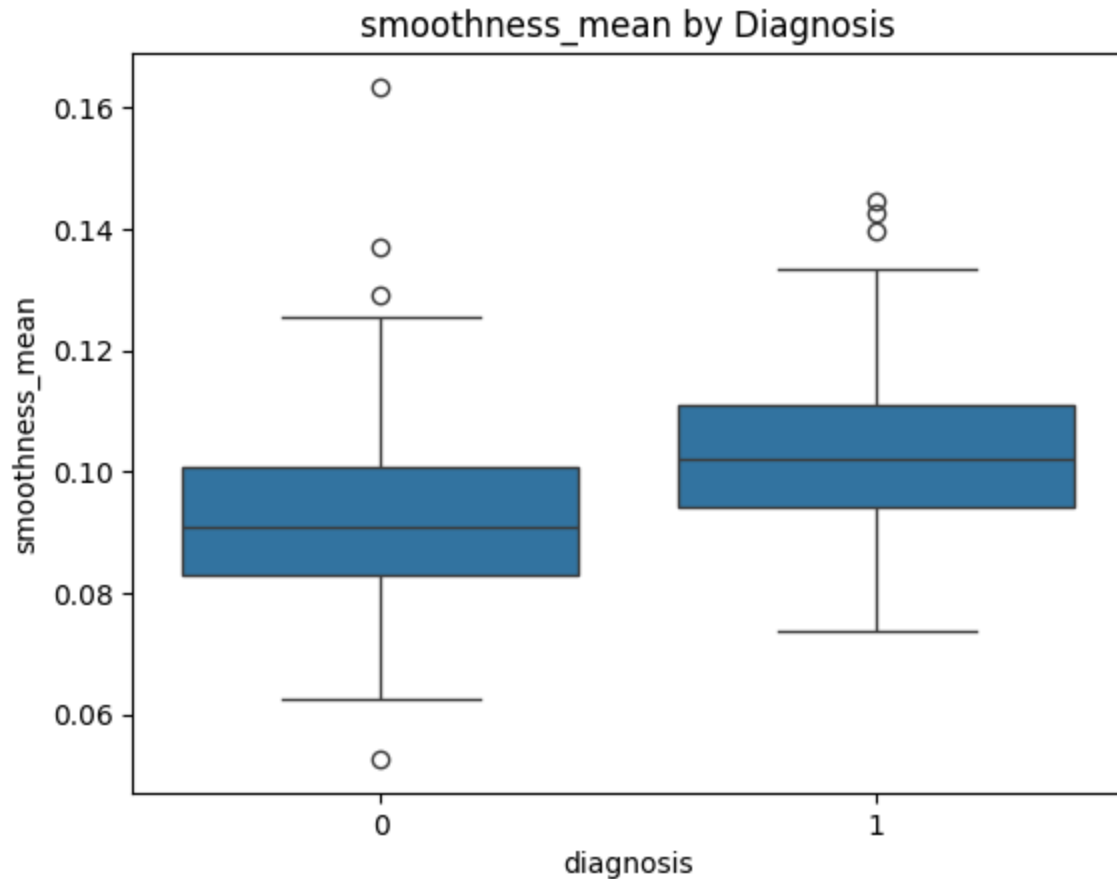
In [10]: `# correlation heatmap`  
`plt.figure(figsize=(12,10))`  
`sns.heatmap(data.corr(), cmap='coolwarm')`  
`plt.title('Feature Correlation Heatmap')`  
`plt.show()`



```
In [11]: # boxplots of selected features
top_features = ['radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean']
for feature in top_features:
    sns.boxplot(x='diagnosis', y=feature, data=data)
    plt.title(f'{feature} by Diagnosis')
    plt.show()
```







### Section 3: Preprocessing

```
In [ ]: from sklearn.model_selection import train_test_split, GridSearchCV
        from sklearn.preprocessing import StandardScaler

        # split features and target
        X = data.drop('diagnosis', axis=1)
        y = data['diagnosis']

        # standardize the features
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        # train-test split
        X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2)
```

### Section 4: Model Building and Evaluation

```
In [ ]: from sklearn.linear_model import LogisticRegression
        from sklearn.neighbors import KNeighborsClassifier
        from sklearn.tree import DecisionTreeClassifier
        from sklearn.ensemble import RandomForestClassifier
        from sklearn.svm import SVC
        from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

        # define models
        models = {
```

```

'Logistic Regression': LogisticRegression(max_iter=1000),
'K-Nearest Neighbors': KNeighborsClassifier(),
'Decision Tree': DecisionTreeClassifier(),
'Random Forest': RandomForestClassifier(),
'Support Vector Machine': SVC(probability=True)
}

# train and eval models
model_results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:,1]
    accuracy = model.score(X_test, y_test)
    roc_auc = roc_auc_score(y_test, y_prob)
    report = classification_report(y_test, y_pred, output_dict=True)
    f1 = report['weighted avg']['f1-score']
    model_results.append({
        'Model': name,
        'Accuracy': accuracy,
        'F1 Score': f1,
        'ROC AUC': roc_auc
    })
    print(f'\n==== {name} ====')
    print(classification_report(y_test, y_pred))
    print('ROC-AUC:', roc_auc_score(y_test, y_prob))
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d')
    plt.title(f'Confusion Matrix: {name}')
    plt.show()

```

```

==== Logistic Regression ====
              precision    recall  f1-score   support

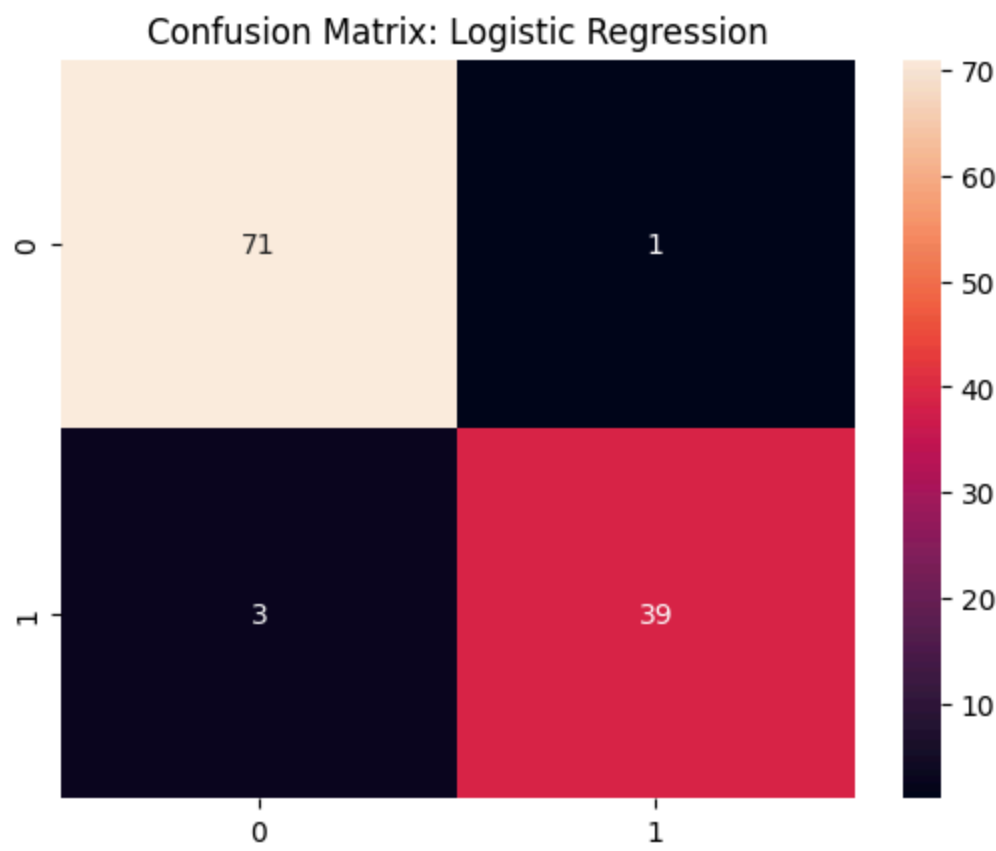
      0               0.96       0.99       0.97         72
      1               0.97       0.93       0.95         42

 accuracy               0.96         114
 macro avg              0.97         114
weighted avg              0.97         114

```

ROC-AUC: 0.996031746031746

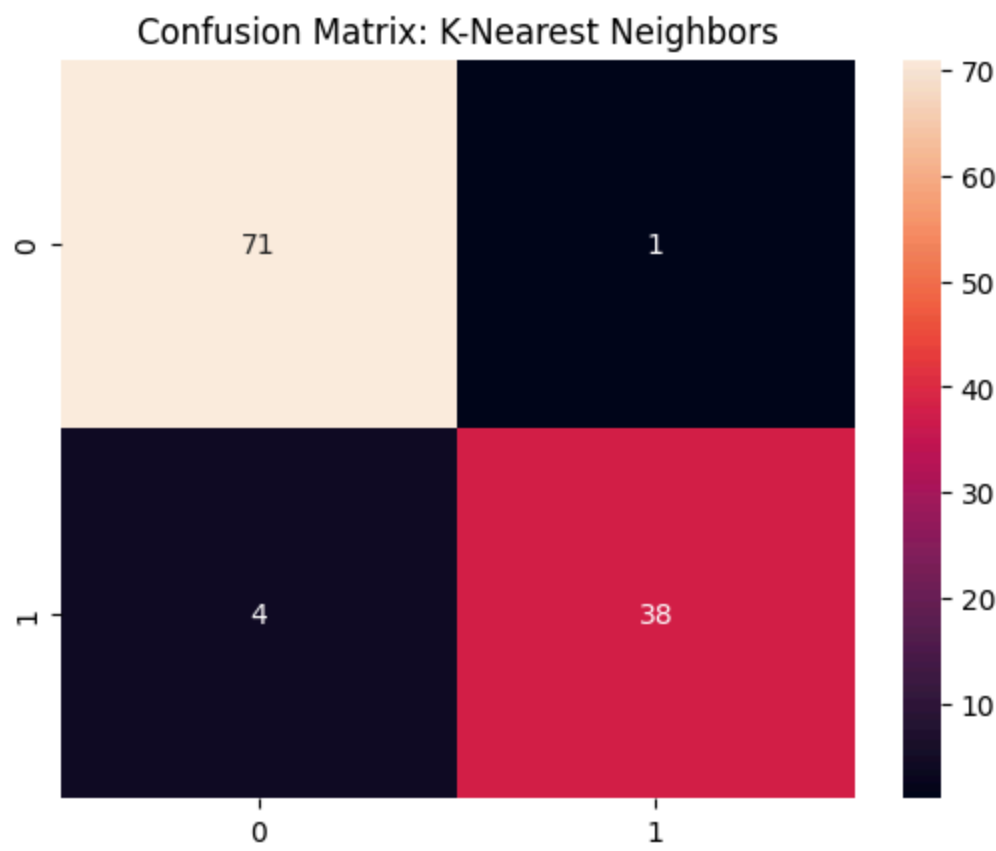




==== K-Nearest Neighbors ====

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.95      | 0.99   | 0.97     | 72      |
| 1            | 0.97      | 0.90   | 0.94     | 42      |
| accuracy     |           |        | 0.96     | 114     |
| macro avg    | 0.96      | 0.95   | 0.95     | 114     |
| weighted avg | 0.96      | 0.96   | 0.96     | 114     |

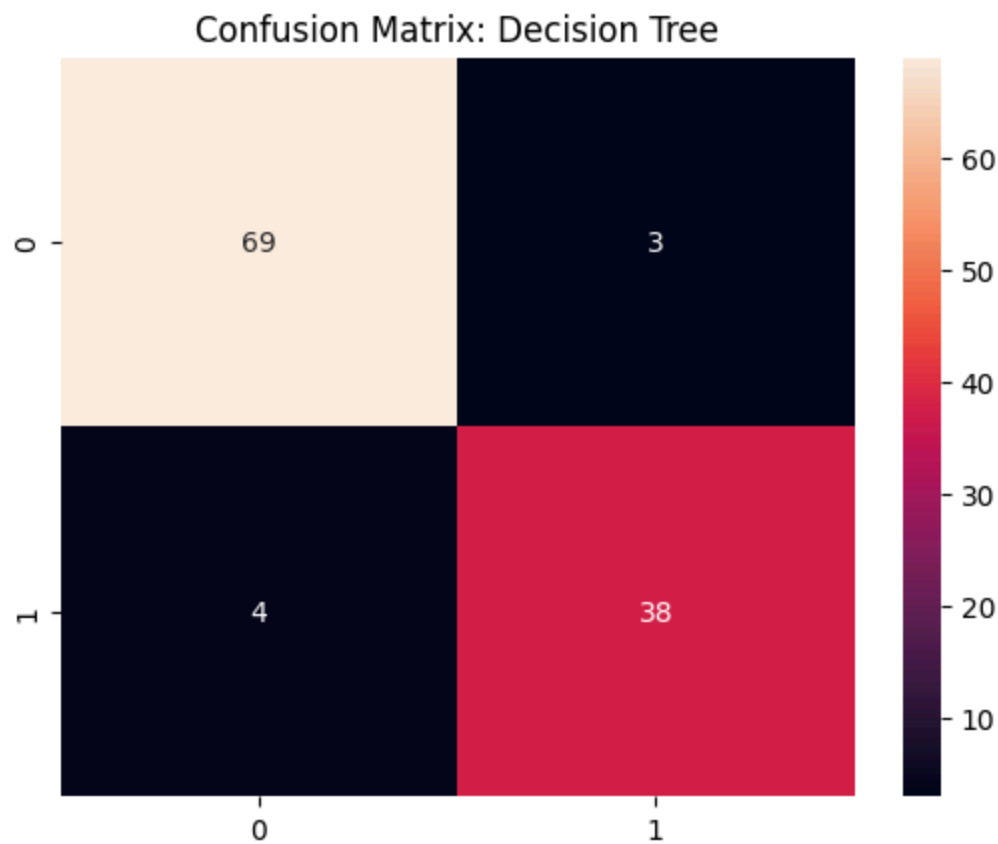
ROC-AUC: 0.9816468253968255



==== Decision Tree ====

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.95      | 0.96   | 0.95     | 72      |
| 1            | 0.93      | 0.90   | 0.92     | 42      |
| accuracy     |           |        | 0.94     | 114     |
| macro avg    | 0.94      | 0.93   | 0.93     | 114     |
| weighted avg | 0.94      | 0.94   | 0.94     | 114     |

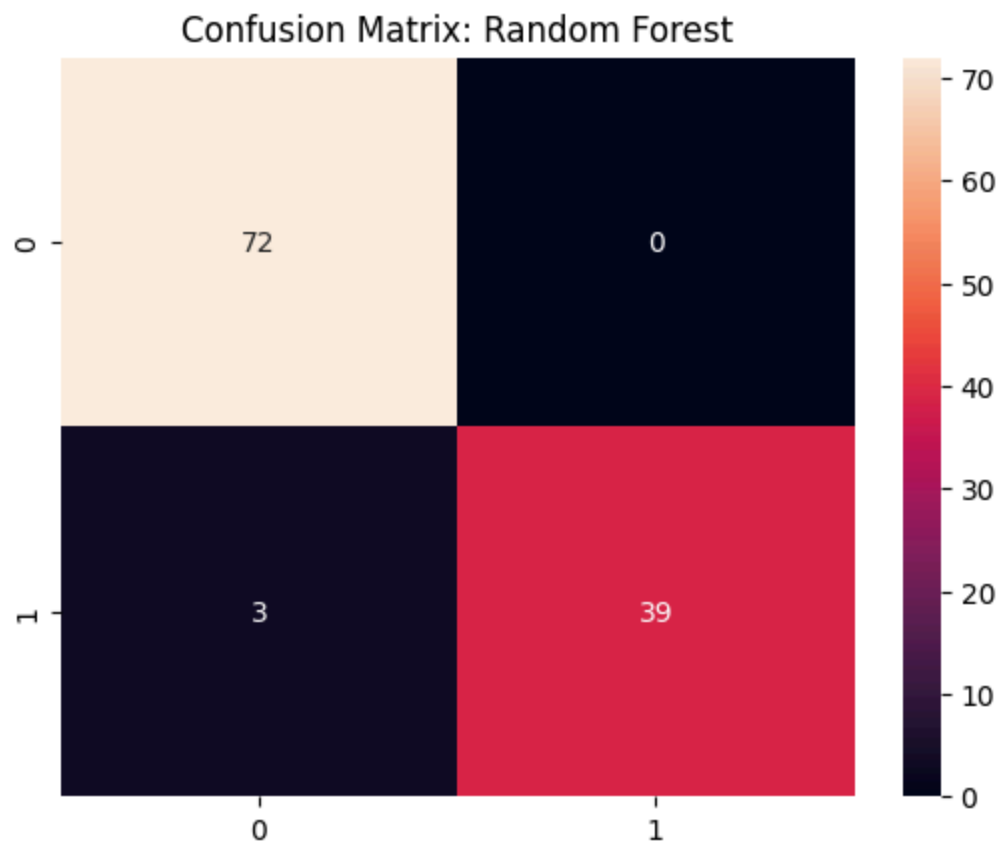
ROC-AUC: 0.9315476190476191



==== Random Forest ====

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.96      | 1.00   | 0.98     | 72      |
| 1            | 1.00      | 0.93   | 0.96     | 42      |
| accuracy     |           |        | 0.97     | 114     |
| macro avg    | 0.98      | 0.96   | 0.97     | 114     |
| weighted avg | 0.97      | 0.97   | 0.97     | 114     |

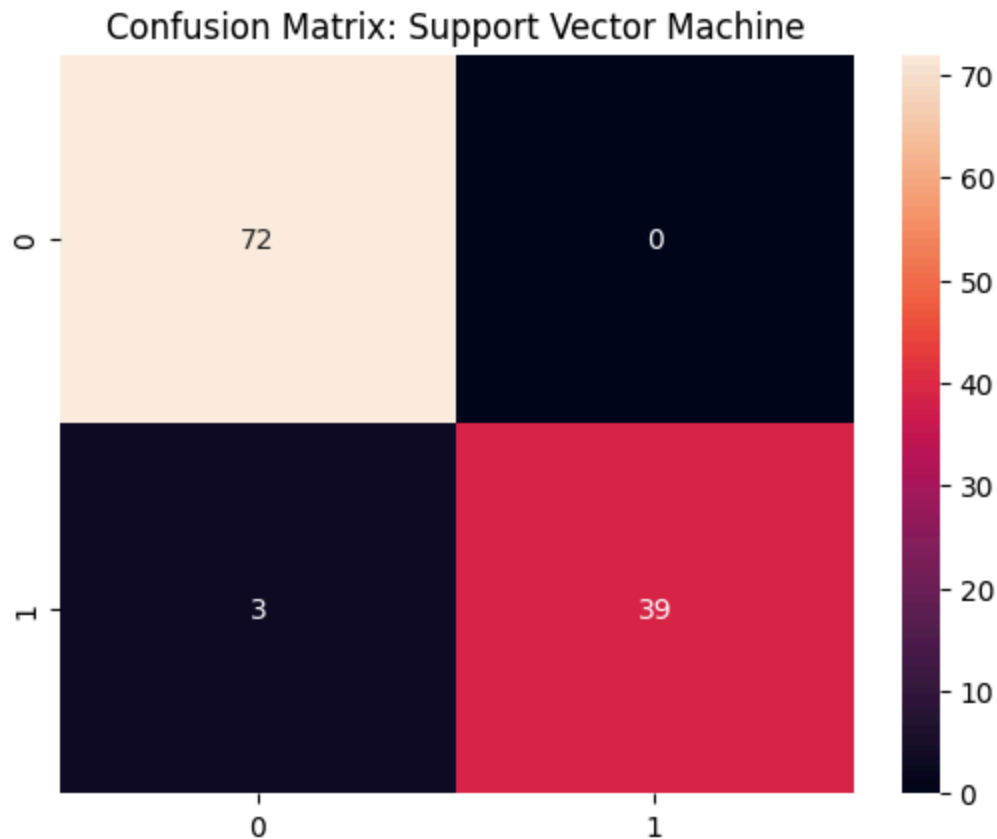
ROC-AUC: 0.9952050264550264



==== Support Vector Machine ====

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.96      | 1.00   | 0.98     | 72      |
| 1            | 1.00      | 0.93   | 0.96     | 42      |
| accuracy     |           |        | 0.97     | 114     |
| macro avg    | 0.98      | 0.96   | 0.97     | 114     |
| weighted avg | 0.97      | 0.97   | 0.97     | 114     |

ROC-AUC: 0.9950396825396826



## Section 5: Model Comparison

```
In [20]: # create a summary table of model results
results_df = pd.DataFrame(model_results)
results_df = results_df.sort_values(by='ROC AUC', ascending=False)
print("\nModel Comparison Table:")
print(results_df)

# plot model comparison
plt.figure(figsize=(10,6))
sns.barplot(data=results_df, x='ROC AUC', y='Model', palette='viridis')
plt.title('Model Comparison by ROC AUC')
plt.xlabel('ROC AUC')
plt.ylabel('Model')
plt.show()
```

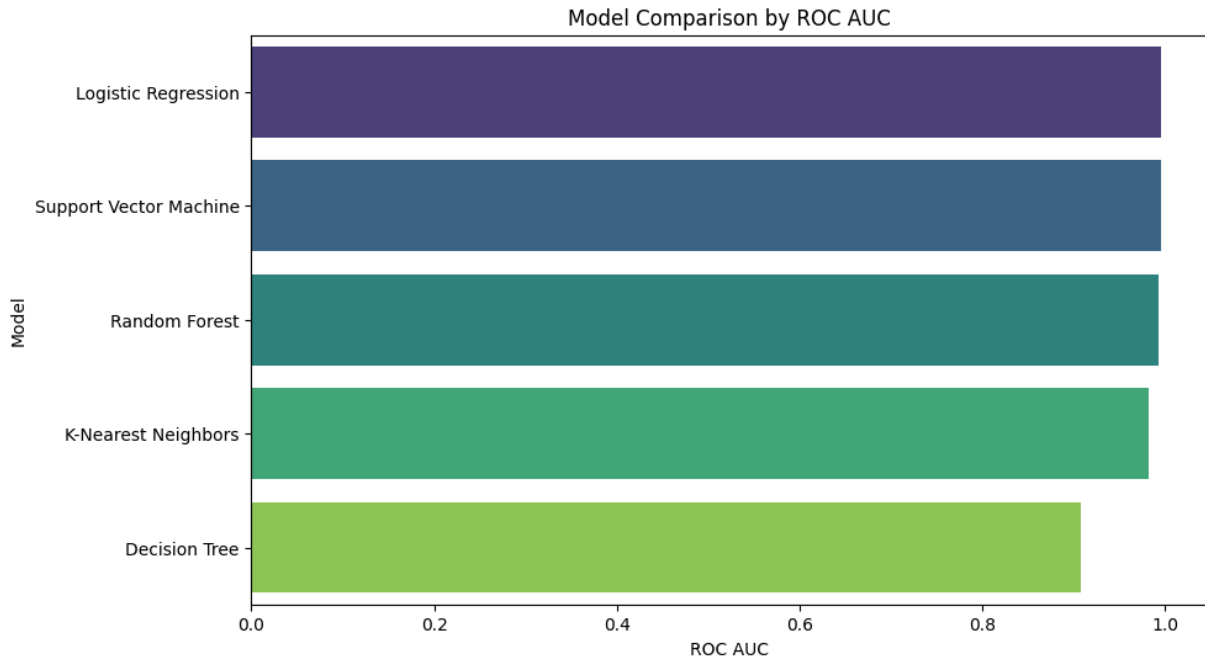
Model Comparison Table:

|   | Model                  | Accuracy | F1 Score | ROC AUC  |
|---|------------------------|----------|----------|----------|
| 0 | Logistic Regression    | 0.964912 | 0.964725 | 0.996032 |
| 4 | Support Vector Machine | 0.973684 | 0.973465 | 0.995040 |
| 3 | Random Forest          | 0.964912 | 0.964509 | 0.992725 |
| 1 | K-Nearest Neighbors    | 0.956140 | 0.955776 | 0.981647 |
| 2 | Decision Tree          | 0.921053 | 0.920396 | 0.907738 |

```
/var/folders/3k/dwgmpl79529bv_2x3vkrv60r0000gn/T/ipykernel_18704/3606928291.py:9: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=results_df, x='ROC AUC', y='Model', palette='viridis')
```



## Section 6: Conclusion and Summary

In this project, we applied multiple supervised learning models to classify breast cancer tumors as malignant or benign using features extracted from digitized images of fine needle aspirates

Five models were evaluated:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)

Based on the evaluation metrics, the Support Vector Machine achieved the best overall performance:

- Accuracy: 97.4%
- F1 Score: 0.973
- ROC AUC: 0.995 This suggests SVM offered the most balanced classification between malignant and benign cases.

Logistic Regression and Random Forest also performed competitively, each achieving ROC AUC > 0.99, indicating excellent separation between classes. These models are strong alternatives, with Logistic Regression being more interpretable and Random Forest offering feature importance insights.

The Decision Tree model underperformed relative to others (ROC AUC = 0.91), likely due to overfitting or lack of tuning. KNN performed well, but slightly lower in recall for malignant cases compared to SVM and Random Forest.

The top 10 most important features identified by the Random Forest model were plotted, with 'worst perimeter', 'mean concave points', and 'worst radius' emerging as the most influential in classification.

Future work could include:

- Hyperparameter tuning using GridSearch or RandomizedSearchCV
- Cross-validation for more robust model comparisons
- Use of interpretability tools like SHAP or LIME to explain model decisions
- Deployment of the best model in a simple web app for demo or real-time inference

Overall, this project demonstrates that traditional supervised learning models, especially SVM and ensemble methods, are highly effective for binary medical classification tasks using structured feature data.